

How AI Changes Software Development

Shubam Chaudhari | June 11, 2026 |

The Big Shift

AI has changed the unit of work in software development from writing code to describing intent. Instead of typing every line, engineers describe what they want in natural language, review what the AI produces, and iterate. The bottleneck moves from typing speed and syntax knowledge to clear thinking: understanding the problem, specifying requirements precisely, and judging whether the output is correct. This is why documents like PRDs matter more than ever — a well-written requirement is now nearly executable.

The Tool Landscape

- End-to-end agents (Replit Agent): take a prompt like “build a habit tracker with login” and scaffold the frontend, backend, database, and deployment in one environment. Best for going from zero to a working app fast.
- Conversational assistants (ChatGPT, Claude): used for planning, explaining unfamiliar code, generating individual components or APIs, writing documentation, and debugging through dialogue. Claude is also widely used through Claude Code, an agent that works directly in a codebase.
- AI-native editors (Cursor): embed AI inside the IDE so it can read the whole project, autocomplete multi-line edits, and make coordinated changes across many files. Best for working inside an existing codebase.

How AI Is Used Across the Stack

- Prompt engineering: the core skill. Good prompts state the goal, constraints, tech stack, and examples (“React + Tailwind, mobile-first, REST API in Express”). Vague prompts produce generic code; specific prompts produce usable code. Iterating on prompts replaces much of the trial-and-error of manual coding.
- UI components: developers describe a screen (“a responsive login form with validation and error states”) and get working React/Tailwind components, then refine spacing, color, and behavior through follow-up prompts.
- Backend APIs: AI generates REST endpoints with routing, input validation, and error handling from a description of the resources (e.g., the POST /register, GET /habits endpoints in our project spec).
- Database schemas: given the entities and relationships, AI proposes tables, columns, keys, and indexes (Users → Habits → Habit Entries) and writes the SQL or ORM models.
- Debugging: pasting an error message or stack trace yields an explanation of the cause and a suggested fix — often faster than searching documentation or Stack Overflow.

What Changes — and What Doesn't

Prototypes that took weeks now take hours, small teams can ship full-stack products, and the boundary between PM, designer, and engineer blurs because anyone who can specify a product clearly can build a version of it. Adoption reflects this: in Stack Overflow's 2025 Developer Survey, 84% of developers said they use or plan to use AI tools — yet 46% don't trust the accuracy of the output. That tension defines what doesn't change: AI code must be reviewed. It can produce insecure patterns (e.g., unhashed passwords), invented APIs, or code that runs but doesn't match requirements. Humans still own the problem definition, the architecture, security, and the judgment about whether the product is right. The developer's role shifts from author to editor — and the best builders are the ones who understand the product, not just the code.

Sources:

Stack Overflow – 2025 Developer Survey, AI section (adoption & trust data): <https://survey.stackoverflow.co/2025/ai>

Replit – Replit Agent Documentation: <https://docs.replit.com/replitai/agent>

Anthropic – Claude Code Documentation: <https://docs.claude.com/en/docs/claude-code/overview>

OpenAI – ChatGPT: <https://openai.com/index/chatgpt/>